LESSON 2

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Traffic Lights

Control three LEDs, make them take turns, and write your own function.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How to control **several pins** at once
- How to plan a **sequence** of steps
- How to write your own **function**
- How an **array** and a **for loop** save typing

SESSION PLAN

0:00	Warm-up: how does a real traffic light work?
0:10	Build the three-LED circuit
0:30	Sketch 1: **L02_TrafficLight**
0:55	Short break
1:00	Sketch 2: **L02_LightFunction** (your own function)
1:20	Sketch 3: **L02_Chase** (arrays and for loops)
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ 1 red, 1 yellow and 1 green LED
- ☐ 3 resistors, 220 Ω (red-red-brown)
- ☐ 4 jumper wires

1. Warm-up: think like a traffic engineer

A traffic light is a little computer that follows a **sequence**: the same steps, in the same order, forever. Before you write any code, write down the steps. Number them, and say which lights are on and for how long.

Plan your traffic light

Write each step: which colours are on, and how many seconds it lasts.

_____

_____

_____

In this lesson we will use **red** → **green** → **yellow** → **back to red**. Some countries do it differently. You'll get to change it in a challenge.

2. Build the circuit

This is the same idea as the LED from Lesson 1, done three times. Each LED gets its own pin and its own resistor. All three short legs share the **–** rail.

✓	From	To
☐	ESP32 pin **GND**	breadboard **–** rail
☐	ESP32 pin **25**	220 Ω resistor → **long leg** of the **red** LED
☐	ESP32 pin **26**	220 Ω resistor → **long leg** of the **yellow** LED
☐	ESP32 pin **27**	220 Ω resistor → **long leg** of the **green** LED
☐	**short legs** of all 3 LEDs	**–** rail

Keep it tidy

Put the LEDs in a line, red at the top, like a real traffic light. Leave a gap between them so the legs don't touch.

3. Sketch 1: A traffic light

File → **New Sketch**, type this in, and save it as **L02_TrafficLight**. Upload it.

Type this in → save it as **L02_TrafficLight** — 32 lines

```
// Lesson 2: Traffic light
// Red LED on 25, yellow on 26, green on 27.

const int RED = 25;
const int YELLOW = 26;
const int GREEN = 27;

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(YELLOW, OUTPUT);
  pinMode(GREEN, OUTPUT);
}

void loop() {
  // Stop
  digitalWrite(RED, HIGH);
  digitalWrite(YELLOW, LOW);
  digitalWrite(GREEN, LOW);
  delay(3000);

  // Go
  digitalWrite(RED, LOW);
  digitalWrite(YELLOW, LOW);
  digitalWrite(GREEN, HIGH);
```

```
  delay(3000);

  // Get ready to stop
  digitalWrite(RED, LOW);
  digitalWrite(YELLOW, HIGH);
  digitalWrite(GREEN, LOW);
  delay(1000);
}
```

How it works

Code	What it means
`const int RED = 25;`	Give pin 25 a name we can remember
`pinMode(RED, OUTPUT);`	Each LED pin needs its own `pinMode`
three `digitalWrite` lines	Set **every** light for this step, even the ones that stay off
`delay(3000);`	Hold this step for 3 seconds

> **Big idea: the computer does exactly what you say**
>
> Why do we switch off lights that are already off? Because each step should say *exactly* what the light looks like. Then you can change the order of the steps and nothing goes wrong.

> **Try it**
>
> 1. Make the green light last twice as long.
> 2. Add a fourth step: all three lights off for half a second.

4. Sketch 2: Write your own function

Sketch 1 has lots of lines that look almost the same. Programmers hate typing the same thing again and again! A **function** is a set of instructions with a name. We'll make one called `lights` that sets all three LEDs and then waits.

File → Save As… and name it **L02_LightFunction**. Then change your code to this:

Type this in → save it as **L02_LightFunction** — 25 lines

```
// Lesson 2: Traffic light with our own function
// Same wiring as L02_TrafficLight.

const int RED = 25;
const int YELLOW = 26;
const int GREEN = 27;

void lights(int r, int y, int g, int wait) {
  digitalWrite(RED, r);
  digitalWrite(YELLOW, y);
  digitalWrite(GREEN, g);
```

```
  delay(wait);
}

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(YELLOW, OUTPUT);
  pinMode(GREEN, OUTPUT);
}

void loop() {
  lights(HIGH, LOW, LOW, 3000);   // stop
  lights(LOW, LOW, HIGH, 3000);   // go
  lights(LOW, HIGH, LOW, 1000);   // get ready to stop
}
```

Code	What it means
`void lights(int r, int y, int g, int wait)`	Make a function called `lights`. It needs 4 numbers, called **parameters**
`digitalWrite(RED, r);`	Use whatever was passed in as `r` (`HIGH` or `LOW`)
`lights(HIGH, LOW, LOW, 3000);`	**Call** the function: red on, yellow off, green off, wait 3 s
`void`	This function does something but doesn't give back an answer

The light works exactly the same as before, but `loop()` is now 3 lines long and easy to read.

> **Same order every time**
>
> The values go into the parameters **in order**. In `lights(LOW, HIGH, LOW, 1000)` the first value goes to `r`, the second to `y`, the third to `g`, and the last to `wait`.

5. Sketch 3: Light chase

Now we'll stop being a traffic light and make a running light, like on a game show. **File → New Sketch**, save it as **L02_Chase**.

Type this in → save it as **L02_Chase** — 19 lines

```
// Lesson 2: Light chase
// Same wiring. A light runs along the LEDs.

int leds[] = {25, 26, 27};
int speed = 150;

void setup() {
  for (int i = 0; i < 3; i++) {
    pinMode(leds[i], OUTPUT);
  }
}
```

```
void loop() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(leds[i], HIGH);
    delay(speed);
    digitalWrite(leds[i], LOW);
  }
}
```

Code	What it means
`int leds[] = {25, 26, 27};`	An **array**: one name holding a list of numbers
`leds[0]`	The **first** item in the list (25). Counting starts at **0**!
`for (int i = 0; i < 3; i++)`	Repeat the `{ }` block with `i` = 0, then 1, then 2
`i++`	Add 1 to `i`

Big idea: loops do the boring work

The `for` loop in `setup()` runs `pinMode` three times for you. With 30 LEDs you would only change the `3` and the array, not write 30 lines.

Try it

1. Change `speed` to `50`. Then `500`.
2. Delete the line `digitalWrite(leds[i], LOW);`. What happens? Why?

6. Challenges

Challenge 1: UK traffic light

In the UK, red and yellow come on **together** before green. Change **L02_LightFunction** to do red → red + yellow → green → yellow.

Challenge 2: Night mode

At night some traffic lights just flash yellow. Make a sketch that flashes the yellow light on and off, with the others off.

Challenge 3: Bouncing light

Change **L02_Chase** so the light goes 25 → 26 → 27 → 26 → 25 → … and bounces back and forth.
Hint: add a second `for` loop that counts down with `i--`.

Bonus: All together now

Make all three LEDs blink together 3 times, then do one chase, then repeat. Write a function for each pattern.

7. Quiz

Check what you learned

1. What is a **function**, and why is it useful?

2. In `int leds[] = {25, 26, 27};` what is `leds[2]` ?

3. How many times does `for (int i = 0; i < 5; i++)` repeat?

Tidy up: unplug the USB cable. Keep your three LEDs on the breadboard: you'll use them again in Lesson 3.